

Práctica de Clase 3 (Hito 3 del TPO): Implementación del Modelo de Dominio en Java

Tema: Traducción de Diagramas de Clases UML a Código Java, Uso de Eclipse IDE

Herramienta: Eclipse IDE for Java Developers

Objetivo

El objetivo de este Hito es traducir el Diagrama de Clases UML que diseñaron en el Hito 2 a una base de código Java funcional y bien estructurada. Al final de esta práctica, tendrán un proyecto de Eclipse que representa la columna vertebral de su sistema LogiSmart, listo para que en futuras clases le agreguemos lógica de negocio y patrones de diseño.

Esta es una tarea de **traducción y atención al detalle**. El objetivo no es implementar la lógica compleja de los métodos, sino crear la **estructura** correcta de clases, atributos y relaciones en código.

Contexto: Del Plano a la Construcción

En el Hito 2, fueron los arquitectos. Crearon el plano (Diagrama de Clases). Ahora, son los constructores. Van a tomar ese plano y levantar la estructura del edificio. Cada "caja" en su diagrama se convertirá en un archivo `.java`. Cada "línea" se convertirá en un atributo o en una palabra clave `extends`.

Actividades

Actividad 1: Configuración del Proyecto en Eclipse

Objetivo: Crear y configurar un nuevo proyecto Java en Eclipse para albergar el código de LogiSmart.

Proceso Paso a Paso:

1. **Abrir Eclipse IDE.**
2. **Crear un nuevo Proyecto Java:**
 - Vayan a **File > New > Java Project**.
 - En **Project name**, escriban **LogiSmartTPO_GrupoXX** (reemplacen XX con su número de grupo).
 - Asegúrense de que la opción **Use default JRE** esté seleccionada y sea una versión reciente (ej: JavaSE-11 o superior).
 - Dejen la opción **Create separate folders for sources and class files** marcada.
 - Hagan clic en **Finish**.
3. **Crear la Estructura de Paquetes:**
 - En el **Package Explorer** (panel izquierdo), expandan su nuevo proyecto.
 - Hagan clic derecho sobre la carpeta **src**.
 - Seleccionen **New > Package**.
 - En el campo **Name**, escriban **com.logismart.dominio**.
 - Hagan clic en **Finish**.

Entregable Parcial: Una captura de pantalla de su Eclipse mostrando la estructura del proyecto con el paquete creado.

Actividad 2: Implementación de Clases y Atributos

Objetivo: Crear un archivo **.java** por cada clase de su diagrama del Hito 2 y definir sus atributos.

Proceso Paso a Paso:

1. **Por cada clase en su diagrama UML:**
 - Hagan clic derecho en el paquete **com.logismart.dominio**.
 - Seleccionen **New > Class**.
 - En **Name**, escriban el nombre de la clase (ej: **Cliente**, **Envio**, **Ruta**).
 - Hagan clic en **Finish**.

2. Dentro de cada archivo de clase recién creado:

- **Definan los atributos** que especificaron en su diagrama. Recuerden la sintaxis de Java: **visibilidad tipo nombre**;
- **Apliquen la encapsulación**: Todos los atributos deben ser **private**.

Ejemplo (para la clase **Vehiculo**):

```
package com.logismart.dominio;

public class Vehiculo {

    // Atributos (privados - encapsulación)
    private String id;
    private String patente;
    private String marca;
    private String modelo;
    private int anio;
    private double capacidadPeso;
    private double capacidadVolumen;
    // ... y todos los demás atributos de su diagrama
}
```

Entregable Parcial: El código de al menos 3 de sus clases principales con todos sus atributos definidos.

Actividad 3: Creación de Constructores, Getters y Setters

Objetivo: Darle a sus clases la capacidad de ser instanciadas (con constructores) y de acceder a su estado de forma controlada (con getters y setters).

Proceso Paso a Paso:

1. Para cada clase:

- **Crear un Constructor:** Definan un constructor público que reciba como parámetros los datos esenciales para crear un objeto de esa clase. Usen la palabra clave **this** para asignar los valores a los atributos.
- **Generar Getters y Setters:**
 - Hagan clic derecho dentro del editor de la clase.
 - Vayan a **Source > Generate Getters and Setters....**
 - Seleccionen todos los atributos y hagan clic en **Generate**.

Ejemplo (continuando con la clase `Vehiculo`):

```
// ... (atributos)

// Constructor

public Vehiculo(String id, String patente, String marca, int anio, double capacidadPeso) {
    this.id = id;
    this.patente = patente;
    this.marca = marca;
    this.anio = anio;
    this.capacidadPeso = capacidadPeso;
}

// Getters y Setters (generados automáticamente)

public String getId() {
    return id;
}

public void setId(String id) {
    this.id = id;
}

public String getPatente() {
    return patente;
}

// ... etc. para todos los atributos
```

Entregable Parcial: El código completo de una de sus clases mostrando el constructor, los getters y los setters.

Actividad 4: Implementación de Relaciones

Objetivo: Traducir las líneas y flechas de su diagrama UML (relaciones) a código Java.

Proceso Paso a Paso:

1. Para cada relación de Asociación o Composición (1 a 1):

- En la clase que "tiene" a la otra, agreguen un **atributo** cuyo tipo sea la clase relacionada.
- **Ejemplo:** Si `Ruta` tiene un `Vehiculo`, en la clase `Ruta` agreguen: `private Vehiculo vehiculoAsignado;`

2. Para cada relación de Asociación o Composición (1 a Muchos):

- Agreguen un atributo que sea una **Colección**, como `List`.
- **Ejemplo:** Si `Ruta` se compone de muchos `PuntosDeEntrega`, en la clase `Ruta` agreguen:
`private List<PuntoDeEntrega> puntosDeEntrega;`
- No olviden importar `java.util.List` y `java.util.ArrayList`.
- En el constructor, inicialicen la lista: `this.puntosDeEntrega = new ArrayList<>();`

3. Para cada relación de Herencia:

- En la clase hija, usen la palabra clave `extends` seguida del nombre de la clase padre.
- **Ejemplo:** `public class Cliente extends Usuario { ... }`
- En el constructor de la clase hija, la primera línea **debe** ser una llamada al constructor de la clase padre usando `super(...)`.

Ejemplo (Clase `Cliente` que hereda de `Usuario`):

```
package com.logismart.dominio;
```

```
public class Cliente extends Usuario {
```

```
    private String razonSocial;
```

```
    public Cliente(String id, String email, String password, String razonSocial) {
```

```
        // 1. Llamar al constructor del padre (Usuario)
```

```
        super(id, email, password);
```

```
        // 2. Inicializar atributos propios
```

```
        this.razonSocial = razonSocial;
```

```
    }
```

```
    // Getters y setters para razonSocial...
```

```
}
```

Entregable Parcial: El código de dos clases que demuestren una relación de composición/asociación y el código de una jerarquía de herencia.

Entregable Final

Un archivo ZIP con su proyecto de Eclipse completo. Para exportar el proyecto:

1. En el **Package Explorer**, hagan clic derecho sobre su proyecto (**LogiSmartTPO_GrupoXX**).
2. Seleccionar **Export...**
3. En la ventana, elijan **General > Archive File**.
4. Hagan clic en **Next**.
5. Asegúrense de que su proyecto esté seleccionado.
6. En **To archive file**, elegir dónde guardar el archivo ZIP y denle un nombre (ej: **HITO_3_GRUPO_XX.zip**).
7. Hagan clic en **Finish**.

El proyecto debe contener:

- ☐ La estructura de paquetes **com.logismart.dominio**.
- ☐ Un archivo **.java** por cada clase de su diagrama del Hito 2.
- ☐ Todas las clases deben tener sus atributos privados.
- ☐ Todas las clases deben tener al menos un constructor.
- ☐ Todas las clases deben tener getters y setters para sus atributos.
- ☐ Todas las relaciones (asociación, composición, herencia) deben estar correctamente implementadas en el código.
- ☐ El proyecto debe compilar sin errores (no debe haber líneas rojas de error en Eclipse).